Hyper-V migration runbook

Step-by-step runbook to rebuild the TF-R Django app on a fresh Ubuntu Server VM in Hyper-V, transfer the PostgreSQL database and uploaded media from the current server, set every required environment variable, and serve it over internal DNS with an internal-CA TLS certificate.

Tube Fab Roman Engineering — TF-R employee portal

Generated from the approved plan. Placeholder hostnames, IP addresses and subnets must be replaced with real values before use.

Task checklist

- [ ] Gather facts from the current server and required values from the network team (Postgres version, IP, FQDN, DNS servers)
- [ ] Create the Hyper-V Gen 2 VM with the MicrosoftUEFICertificateAuthority Secure Boot template and static memory
- [ ] Install Ubuntu 24.04 LTS, set hostname, set timezone to America/Detroit, configure the static IP via netplan
- [ ] Create the internal DNS A and PTR records and verify resolution from a client
- [ ] Export the PostgreSQL dump and the media tarball from the current server and transfer both to the VM
- [ ] Install PostgreSQL, create the tfapp role and empty database, and load the dump before running any migrations
- [ ] Create the service account, clone the repo, build the virtualenv, and restore the media directory
- [ ] Write /etc/tfapp/tfapp.env with all required variables, set 0640 root:tfapp permissions, and verify Django reads them
- [ ] Run migrate and collectstatic in that order, after the dump is loaded
- [ ] Create and enable the tfapp gunicorn systemd unit bound to 127.0.0.1:8000
- [ ] Generate the CSR with SANs, submit it to the internal AD CS, and install the certificate and chain
- [ ] Configure nginx for TLS termination, /static/ and /media/ serving, and safe X-Forwarded-For handling

- [ ] Configure ufw to allow 80/443 and restrict SSH while keeping PostgreSQL on localhost
- [ ] Install cron entries for flag_missing_punches and nightly database backups
- [ ] Update TimeclockKioskIP records for the new subnet, or migrate kiosks to token auth
- [ ] Work through the verification checklist including manage.py check --deploy and a kiosk punch test
- [ ] Execute the cutover: freeze, final dump, reload, DNS repoint, smoke test, retain the old server for rollback

TF-R migration to Hyper-V: fresh build runbook

Rebuild on a clean Ubuntu Server 24.04 LTS VM, then move the data across. Nothing from the old disk is carried over except the PostgreSQL dump and the `media/` uploads, which is what makes this approach clean: the resulting server is fully described by this document.

Target architecture, internal only, remote users via corporate VPN, cloudflared retired:

Architecture diagram (Mermaid source)

```
flowchart LR
    users["Staff browsers and kiosk PCs"] --> dns["Internal DNS A record"]
    vpn["Remote users over corporate VPN"] --> dns
    dns --> nginx["nginx :443, internal CA cert"]
    nginx --> gunicorn["gunicorn on 127.0.0.1:8000"]
    nginx --> files["/static/ and /media/ served from disk"]
    gunicorn --> postgres["PostgreSQL on localhost:5432"]
```

Substitute your real values for these placeholders throughout:

- `tfapp.corp.example.com` - the final FQDN
- `10.0.10.50` - the VM's static IP
- `10.0.10.1` - the gateway
- `10.0.10.10`, `10.0.10.11` - internal DNS servers
- `OLDSERVER` - hostname or IP of the current Ubuntu box

Step 1: Gather facts before you build

Run these on the **current** server and save the output. The PostgreSQL major version is the one value that can force a rework later, because a dump from a newer PostgreSQL cannot be loaded into an older one.

```
lsb_release -a
python3 --version
psql --version
sudo -u postgres psql -c "SELECT version();"
sudo -u postgres psql -c "\l"
systemctl list-units --type=service | grep -Ei 'gunicorn|nginx|cloudflared|postgres'
crontab -l; sudo crontab -l
ip -4 addr; ip route
```

Capture the currently exported environment so nothing is lost. If the app runs under systemd, this shows the real values in use:

```
sudo systemctl show gunicorn -p Environment -p EnvironmentFiles
sudo tr '\0' '\n' < /proc/$(pgrep -f 'gunicorn.*tfapp' | head -1)/environ | grep -E 'DJANGO|DB_|EMAIL|SESSION'
```

Also measure what you are moving, so you can size the disk and predict transfer time:

```
du -sh /path/to/tfapp/media
sudo -u postgres psql -d tfapp -c "SELECT pg_size_pretty(pg_database_size('tfapp'));"
```

Request from the network and AD teams in parallel, since these usually have lead time: a static IP or DHCP reservation, the agreed FQDN, the internal DNS server addresses, a firewall rule permitting 443 from user subnets, and confirmation that a `WebServer` certificate template is available on the internal CA.

Step 2: Create the Hyper-V VM

From an elevated PowerShell prompt on the Hyper-V host:

```
New-VM -Name "TFAPP-PROD" -Generation 2 -MemoryStartupBytes 8GB `
  -NewVHDPath "D:\VMs\TFAPP-PROD\tfapp.vhdx" -NewVHDSizeBytes 120GB `
  -SwitchName "Production-vSwitch"

Set-VM -Name "TFAPP-PROD" -ProcessorCount 4 -StaticMemory
Set-VMFirmware -VMName "TFAPP-PROD" -SecureBootTemplate MicrosoftUEFICertificateAuthority
Add-VMDvdDrive -VMName "TFAPP-PROD" -Path "D:\ISO\ubuntu-24.04.x-live-server-amd64.iso"

$dvd = Get-VMDvdDrive -VMName "TFAPP-PROD"
Set-VMFirmware -VMName "TFAPP-PROD" -FirstBootDevice $dvd
Start-VM -Name "TFAPP-PROD"
```

Three settings are deliberate:

- **Secure Boot template.** Generation 2 VMs default to a Microsoft Windows template that refuses to boot Ubuntu. The `MicrosoftUEFICertificateAuthority` template is what makes shim and GRUB validate. Skipping this produces a confusing boot failure with no obvious cause.
- **Static memory, not Dynamic Memory.** PostgreSQL sizes shared buffers once at startup; ballooning memory underneath it degrades performance unpredictably.
- **Generation 2**, so the install is UEFI and you get a modern SCSI boot path.

Retrieve the MAC address for the DHCP reservation request:

```
Get-VMNetworkAdapter -VMName "TFAPP-PROD" | Select-Object MacAddress
```

Step 3: Install Ubuntu and configure the base system

Install Ubuntu Server 24.04 LTS. Choose the minimal install, enable OpenSSH, create your admin user, and skip all snap extras. 24.04 ships Python 3.12, matching the version Dockerfile pins, so the pinned wheels in `requirements.txt` install cleanly.

After first boot:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y linux-tools-virtual linux-cloud-tools-virtual
sudo hostnamectl set-hostname tfapp-prod
sudo timedatectl set-timezone America/Detroit
timedatectl   # confirm
```

The Hyper-V tools packages give you guest IP reporting and clean shutdown integration from the host. The timezone is not cosmetic: tfapp/tfapp/settings.py sets `TIME_ZONE = 'America/Detroit'`, and this is a timeclock and payroll system. A VM left on UTC shifts punch times by four or five hours and pushes late-shift punches into the wrong calendar day, which corrupts attendance occurrences and payroll totals.

Configure the static IP. Find the interface name with `ip -br addr`, then edit `/etc/netplan/01-netcfg.yaml`:

```
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: no
      addresses: [10.0.10.50/24]
      routes:
        - to: default
          via: 10.0.10.1
      nameservers:
        addresses: [10.0.10.10, 10.0.10.11]
        search: [corp.example.com]
```

```
sudo chmod 600 /etc/netplan/01-netcfg.yaml
sudo netplan apply
ip -4 addr show eth0
ping -c2 10.0.10.1 && getent hosts corp.example.com
```

Netplan warns about world-readable config, hence the `chmod`. Apply over SSH with care; a mistake here drops your session and you will need console access from Hyper-V Manager.

Step 4: Internal DNS

No Active Directory domain join is required, so this is purely a DNS record. On a Windows DNS server:

```
Add-DnsServerResourceRecordA -ZoneName "corp.example.com" -Name "tfapp" `
  -IPv4Address "10.0.10.50" -CreatePtr
```

Verify from a client workstation and from the VM itself:

```
Resolve-DnsName tfapp.corp.example.com
```

```
getent hosts tfapp.corp.example.com
```

Lock in the FQDN now. It feeds `DJANGO_ALLOWED_HOSTS`, `DJANGO_CSRF_TRUSTED_ORIGINS`, `DJANGO_SITE_BASE_URL`, and the certificate SAN, so changing it later means touching four things.

Step 5: Export the database and media from the current server

Run on **OLDSERVER**. Take the dump in PostgreSQL's custom format, which is compressed and lets `pg_restore` remap ownership:

```
cd /path/to/tfapp
export PGPASSWORD='<current DB_PASSWORD>'
pg_dump -h localhost -U tfapp -d tfapp -F c -b -v -f ~/tfapp-$(date +%F).dump
unset PGPASSWORD
ls -lh ~/tfapp-*.dump
```

The flags matter: `-F c` is the custom format required for `--no-owner` and `--role` on restore, `-b` includes large objects, and `-v` gives you progress on a database of any size. The repo's tfapp/scripts/backup_db.sh produces an equivalent dump if you would rather use it, since it reads credentials through Django.

Archive the uploads. These are not in git and are not in the database:

```
tar czf ~/tfapp-media-$(date +%F).tar.gz -C /path/to/tfapp media
tar tzf ~/tfapp-media-$(date +%F).tar.gz | head
```

The `media/` tree holds the employee handbook PDF, event images, profile photos, and profile credential documents, per the `upload_to` paths in tfapp/resources/models.py and tfapp/accounts/models.py. Losing it means broken links throughout the app with no way to recover.

Transfer both to the VM:

```
scp ~/tfapp-*.dump ~/tfapp-media-*.tar.gz youruser@10.0.10.50:/tmp/
```

If the two networks cannot reach each other, copy via a USB disk or an intermediate share. Verify integrity after any indirect transfer by comparing `sha256sum` on both ends.

Step 6: Install PostgreSQL and load the data

Install the server. Ubuntu 24.04 provides PostgreSQL 16, which safely accepts dumps from 16 and earlier:

```
sudo apt install -y postgresql postgresql-contrib
systemctl status postgresql --no-pager
psql --version
```

If Step 1 showed the old server running something **newer** than 16, stop and install that major version from the PGDG repository instead, because `pg_restore` cannot read a dump produced by a newer major version:

```
sudo apt install -y curl ca-certificates
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc \
  https://www.postgresql.org/media/keys/ACCC4CF8.asc
echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.asc] \
  https://apt.postgresql.org/pub/repos/apt noble-pgdg main" \
  | sudo tee /etc/apt/sources.list.d/pgdg.list
sudo apt update && sudo apt install -y postgresql-17
```

Create the role and an **empty** database. Use a strong password and record it; it goes into the env file in Step 8:

```
sudo -u postgres createuser --pwprompt tfapp
sudo -u postgres createdb --owner=tfapp --encoding=UTF8 tfapp
sudo -u postgres psql -c "\l tfapp"
```

Now load the dump. Order is critical here:

```
sudo -u postgres pg_restore --no-owner --no-privileges --role=tfapp \
  -d tfapp -v /tmp/tfapp-2026-09-07.dump 2>&1 | tee /tmp/restore.log
```

Do **not** run `manage.py migrate` before this. The dump contains the full schema, so restoring onto an already-migrated database fails with "relation already exists" for every table. The correct sequence is create empty database, restore dump, then migrate to apply anything newer. `--no-owner` and `--role=tfapp` reassign objects to the new role, which matters if the old database was owned by a differently named user.

Some noise in `restore.log` is normal and harmless: warnings about the `public` schema already existing, or about comments on extensions. What matters is that the row counts come out right:

```
sudo -u postgres psql -d tfapp -c "
SELECT 'users' AS table, count(*) FROM attendance_customuser
UNION ALL SELECT 'punches', count(*) FROM timeclock_timeclockentry
UNION ALL SELECT 'kiosk_ips', count(*) FROM timeclock_timeclockkioskip
UNION ALL SELECT 'migrations', count(*) FROM django_migrations;"
```

Compare each number against the same query on OLDSERVER. If a table name differs, list them with `\dt` and adjust. Also confirm the role can actually connect, which catches `pg_hba.conf` and password problems before Django obscures them:

```
PGPASSWORD='<password>' psql -h localhost -U tfapp -d tfapp -c "SELECT current_user, current_database();"
```

The default `pg_hba.conf` on Ubuntu already permits `md5`/`scram` password auth on `127.0.0.1`, so no change is normally needed. Keep PostgreSQL listening on localhost only; there is no reason to expose 5432 on the network.

Step 7: Deploy the application code

Create an unprivileged service account that owns nothing but the app:

```
sudo adduser --system --group --home /opt/tfapp tfapp
sudo apt install -y python3-venv python3-dev build-essential libpq-dev git
```

Clone the repository and build the virtualenv:

```
sudo install -d -o tfapp -g tfapp /opt/tfapp
sudo -u tfapp git clone https://github.com/BackWoodsGinger/tfapp.git /opt/tfapp/app
sudo -u tfapp python3 -m venv /opt/tfapp/venv
sudo -u tfapp /opt/tfapp/venv/bin/pip install --upgrade pip
sudo -u tfapp /opt/tfapp/venv/bin/pip install -r /opt/tfapp/app/tfapp/requirements-prod.txt
```

Use `requirements-prod.txt`, not `requirements.txt`. It pulls in the base pins plus gunicorn, whitenoise, and psycopg 3, which is what a production install needs.

Restore the uploads into place and fix ownership:

```
sudo -u tfapp tar xzf /tmp/tfapp-media-2026-09-07.tar.gz -C /opt/tfapp/app/tfapp
sudo chown -R tfapp:tfapp /opt/tfapp/app/tfapp/media
sudo find /opt/tfapp/app/tfapp/media -type d -exec chmod 755 {} \;
sudo find /opt/tfapp/app/tfapp/media -type f -exec chmod 644 {} \;
ls -la /opt/tfapp/app/tfapp/media
```

nginx runs as `www-data` and must be able to read these files, hence the world-readable modes.

Step 8: Set the environment variables

This replaces the current approach of exporting variables in a shell. A single root-owned file read by both systemd and cron means the web app and the nightly job can never disagree about configuration.

Create the directory and file:

```
sudo install -d -m 750 -o root -g tfapp /etc/tfapp
sudo -e /etc/tfapp/tfapp.env
```

Full contents. Every line is either required or a deliberate production choice:

```
# --- Database (all five are mandatory; no defaults exist in settings.py) ---
DB_NAME=tfapp
DB_USER=tfapp
DB_PASSWORD=<the password you set in Step 6>
DB_HOST=localhost
DB_PORT=5432

# --- Core Django ---
DJANGO_SECRET_KEY=<generate per the command below>
DJANGO_DEBUG=False
DJANGO_ALLOWED_HOSTS=tfapp.corp.example.com,10.0.10.50,localhost,127.0.0.1
DJANGO_CSRF_TRUSTED_ORIGINS=https://tfapp.corp.example.com
DJANGO_SITE_BASE_URL=https://tfapp.corp.example.com

# --- Behind the nginx reverse proxy ---
DJANGO_SECURE_PROXY_SSL_HEADER=1
DJANGO_TRUST_X_FORWARDED_FOR=1
DJANGO_USE_WHITENOISE=1

# --- Sessions (defaults shown; uncomment only to change) ---
# SESSION_COOKIE_AGE=5400
# DJANGO_MAX_SESSIONS_PER_USER=3

# --- Email: omit EMAIL_HOST entirely to keep the console backend ---
# EMAIL_HOST=smtp.corp.example.com
# EMAIL_PORT=587
# EMAIL_USE_TLS=1
# EMAIL_HOST_USER=
# EMAIL_HOST_PASSWORD=
# DEFAULT_FROM_EMAIL=TF-R App <noreply@corp.example.com>
```

Why each of the non-obvious ones is set:

- **The five `DB_*` variables have no defaults.** tfapp/tfapp/settings.py reads them through python-decouple with no fallback, so a missing or misspelled one raises `UndefinedValueError` during settings import. The failure mode is total: gunicorn will not start, `manage.py` will not run, and even `collectstatic` fails. This is the single most common cause of a failed rebuild.
- `DJANGO_DEBUG=False` turns on the security defaults, since `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, and one-year HSTS are all derived from `not DEBUG`. That is correct here precisely because nginx terminates TLS. Leaving `DEBUG=True` in production would also expose tracebacks containing this database password.
- `DJANGO_SECURE_PROXY_SSL_HEADER=1` makes Django trust `X-Forwarded-Proto` from nginx. Without it, Django sees plain HTTP from the proxy, decides the request is insecure, and issues an HTTPS redirect that nginx forwards back, producing an infinite redirect loop. This is the classic symptom of a Django app newly placed behind a proxy.
- `DJANGO_TRUST_X_FORWARDED_FOR=1` is required for kiosk recognition. `get_client_ip` in tfapp/timeclock/kiosk.py defaults to `REMOTE_ADDR`, which behind nginx is always `127.0.0.1`. Without this, every kiosk stops being recognised.
- `DJANGO_USE_WHITENOISE=1` enables compressed static file storage and the WhiteNoise middleware. nginx also serves `/static/` directly, so WhiteNoise is a backstop, but the compressed storage backend is what `collectstatic` uses to produce hashed assets.

Generate the secret key rather than copying the old one:

```
/opt/tfapp/venv/bin/python -c "from django.core.management.utils import get_random_secret_key; print(get_random_sec
```

Rotating it invalidates every existing session and password-reset link, which is exactly what you want during a cutover. Note that the restored `django_session` rows become unusable, so everyone logs in once after the move.

Lock the file down. It contains the database password:

```
sudo chown root:tfapp /etc/tfapp/tfapp.env
sudo chmod 640 /etc/tfapp/tfapp.env
ls -l /etc/tfapp/tfapp.env    # -rw-r----- root tfapp
```

Mode 0640 with group `tfapp` lets the service account read it while keeping it invisible to other users. Verify Django actually resolves everything before going further:

```
sudo -u tfapp bash -c 'set -a; . /etc/tfapp/tfapp.env; set +a;
  cd /opt/tfapp/app/tfapp && /opt/tfapp/venv/bin/python manage.py check'
```

A clean `System check identified no issues` proves the file is readable by `tfapp`, every mandatory variable resolved, and the database credentials parse. If you instead see `UndefinedValueError`, a `DB_*` line is missing or misspelled.

Step 9: Apply migrations and collect static files

Only now, after the dump is loaded:

```
sudo -u tfapp bash -c 'set -a; . /etc/tfapp/tfapp.env; set +a;
  cd /opt/tfapp/app/tfapp && /opt/tfapp/venv/bin/python manage.py migrate'

sudo -u tfapp bash -c 'set -a; . /etc/tfapp/tfapp.env; set +a;
  cd /opt/tfapp/app/tfapp && /opt/tfapp/venv/bin/python manage.py collectstatic --noinput'
```

If the VM is on the same commit as the old server, `migrate` reports no changes to apply, which confirms the restore was complete and consistent. Any migrations it does apply are ones the old server had not reached yet.

Confirm static files landed in `staticfiles/`, the path `STATIC_ROOT` points at and the directory nginx will serve:

```
ls /opt/tfapp/app/tfapp/staticfiles | head
```

Step 10: Run gunicorn under systemd

Create `/etc/systemd/system/tfapp.service`:

```
[Unit]
Description=TF-R Django application
After=network-online.target postgresql.service
Wants=network-online.target
Requires=postgresql.service

[Service]
User=tfapp
Group=tfapp
WorkingDirectory=/opt/tfapp/app/tfapp
EnvironmentFile=/etc/tfapp/tfapp.env
ExecStart=/opt/tfapp/venv/bin/gunicorn tfapp.wsgi:application \
  --bind 127.0.0.1:8000 \
  --workers 3 \
  --timeout 120 \
  --access-logfile /var/log/tfapp/access.log \
  --error-logfile /var/log/tfapp/error.log
Restart=always
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

```
sudo install -d -o tfapp -g tfapp /var/log/tfapp
sudo systemctl daemon-reload
sudo systemctl enable --now tfapp
systemctl status tfapp --no-pager
curl -I http://127.0.0.1:8000/accounts/login
```

`EnvironmentFile` is why Step 8 used `KEY=value` with no `export` and no shell quoting: systemd parses the file itself and does not run it through a shell. Binding to `127.0.0.1` rather than `0.0.0.0` ensures nobody can reach gunicorn directly and bypass TLS and the proxy headers.

One caveat: because `Requires=postgresql.service` is paired with `Restart=always`, a database outage produces a restart loop in the journal rather than a silent failure. Check with `journalctl -u tfapp -f` if the service flaps.

Step 11: Certificate from the internal CA

Generate the key and CSR on the VM so the private key never leaves it:

```
sudo install -d -m 700 /etc/ssl/tfapp
cd /etc/ssl/tfapp
sudo openssl req -new -newkey rsa:2048 -nodes \
  -keyout tfapp.key -out tfapp.csr \
  -subj "/CN=tfapp.corp.example.com/O=Tube Fab Roman/C=US" \
  -addext "subjectAltName=DNS:tfapp.corp.example.com,DNS:tfapp,IP:10.0.10.50"
sudo chmod 600 tfapp.key
```

```
sudo openssl req -in tfapp.csr -noout -text | grep -A1 "Subject Alternative Name"
```

The SAN extension is mandatory. Current Chrome and Edge ignore the Common Name entirely and validate only against SANs, so a CN-only certificate produces `ERR_CERT_COMMON_NAME_INVALID` even when the name matches.

Submit the CSR from a domain-joined Windows machine:

```
certreq -submit -attrib "CertificateTemplate:WebServer" tfapp.csr tfapp.cer
```

If your CA requires manual approval, the request will pend; retrieve it afterwards with `certreq -retrieve <RequestId> tfapp.cer`. Copy `tfapp.cer` plus the issuing and root CA certificates back to the VM and assemble the chain in leaf-then-intermediate order, which is what nginx expects:

```
cd /etc/ssl/tfapp
sudo openssl x509 -inform DER -in tfapp.cer -out tfapp.crt 2>/dev/null || sudo cp tfapp.cer tfapp.crt
sudo bash -c 'cat tfapp.crt intermediate-ca.crt > tfapp-fullchain.crt'
sudo openssl verify -CAfile root-ca.crt -untrusted intermediate-ca.crt tfapp.crt
```

`certreq` may hand back DER rather than PEM; the conversion above covers both. Domain-joined clients already trust your internal root through Group Policy, so no browser warnings and nothing to deploy to workstations.

Step 12: nginx

```
sudo apt install -y nginx
```

Create `/etc/nginx/sites-available/tfapp`:

```
server {
    listen 80;
    server_name tfapp.corp.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    http2 on;
    server_name tfapp.corp.example.com;

    ssl_certificate     /etc/ssl/tfapp/tfapp-fullchain.crt;
    ssl_certificate_key /etc/ssl/tfapp/tfapp.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    client_max_body_size 25M;

    access_log /var/log/nginx/tfapp-access.log;
    error_log  /var/log/nginx/tfapp-error.log;

    location /static/ {
        alias /opt/tfapp/app/tfapp/staticfiles/;
        expires 30d;
        access_log off;
    }

    location /media/ {
        alias /opt/tfapp/app/tfapp/media/;
    }

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host              $host;
```

```
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_redirect off;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/tfapp /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

Three choices worth understanding:

- `X-Forwarded-For` **is set to** `$remote_addr`**, not** `$proxy_add_x_forwarded_for`**.** Kiosk authentication trusts the leftmost value in that header, and the usual append form preserves whatever the client sent. A user could then forge a registered kiosk IP and punch without a PIN. Overwriting the header closes that hole.
- `client_max_body_size 25M` because the app accepts handbook PDF and credential uploads; nginx's 1 MB default would reject them with a 413 before Django ever sees the request.
- `/media/` **is served explicitly.** tfapp/tfapp/urls.py only wires up media under `DEBUG`, and WhiteNoise handles static files but never media. So with `DEBUG=False` and no web server in front, uploads return 404. nginx serving `/media/` directly is what fixes that.

Be aware that this `/media/` block makes every upload readable by anyone on the network who knows or guesses a URL, with no authentication, including files under `profile_credentials/`. That is the same exposure the app has today, so it is not a regression, but if those documents are sensitive the fix is an authenticated view using `X-Accel-Redirect`. I would treat that as a follow-up rather than part of the migration.

Step 13: Firewall

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 10.0.20.0/24 to any port 22 proto tcp comment 'admin SSH'
sudo ufw allow 80/tcp comment 'HTTP redirect'
sudo ufw allow 443/tcp comment 'HTTPS'
sudo ufw enable
sudo ufw status verbose
```

Restrict SSH to your admin subnet and confirm you are connecting from it before enabling, or you will lock yourself out and need the Hyper-V console. PostgreSQL and gunicorn both listen on localhost and need no rules.

Step 14: Scheduled jobs

tfapp/timeclock/management/commands/flag_missing_punches.py documents a nightly 2:30 AM run. If it is lost in the move, incomplete punches quietly stop being flagged and nobody notices until payroll. Install both jobs in root's crontab via `sudo crontab -e`:

```
30 2 * * * set -a && . /etc/tfapp/tfapp.env && set +a && cd /opt/tfapp/app/tfapp && /opt/tfapp/venv/bin/python mana
0  2 * * * set -a && . /etc/tfapp/tfapp.env && set +a && cd /opt/tfapp/app/tfapp && ./scripts/backup_db.sh >> /var/
```

Cron runs with a nearly empty environment, which is why each line sources the env file explicitly; this is the usual reason a job that works interactively fails under cron. Test before trusting the schedule, using the command's own dry-run flag:

```
sudo bash -c 'set -a; . /etc/tfapp/tfapp.env; set +a;
  cd /opt/tfapp/app/tfapp && /opt/tfapp/venv/bin/python manage.py flag_missing_punches --dry-run --no-email'
sudo bash -c 'set -a; . /etc/tfapp/tfapp.env; set +a;
```

```
  cd /opt/tfapp/app/tfapp && ./scripts/backup_db.sh'
ls -lh /opt/tfapp/app/tfapp/backups/
```

Cross-check against the old server's crontab from Step 1 in case other jobs exist. Backups self-prune after 30 days; point them at a network share if one is available, since a backup on the same VHDX does not survive losing the VM.

Step 15: Update kiosk records for the new subnet

This data lives in the database, not in config, and is the item most likely to be forgotten:

- `TimeclockKioskIP` in tfapp/timeclock/models.py matches the client IP by exact string, with no CIDR or wildcard support and a cap of five records. Every kiosk that moves to the new subnet needs its row updated in Django admin under Timeclock kiosk IPs, or it silently loses PIN-less punching.
- Never register `127.0.0.1` or the nginx host itself as a kiosk IP. If the app ever falls back to `REMOTE_ADDR`, every user on the site would be treated as a kiosk and could punch as anyone without a PIN.
- `TimeclockKioskToken` authenticates by long-lived cookie instead, so it is the better choice for any kiosk whose address may change again.
- Verify by signing in as a staff user and visiting `/timeclock/`, which displays the detected client IP and whether kiosk auth resolved by IP, by token, or not at all.

Step 16: Verification

Service and configuration level:

```
systemctl status tfapp nginx postgresql --no-pager
sudo -u tfapp bash -c 'set -a; . /etc/tfapp/tfapp.env; set +a;
  cd /opt/tfapp/app/tfapp && /opt/tfapp/venv/bin/python manage.py check --deploy'
curl -I http://tfapp.corp.example.com          # expect 301 to https
curl -I https://tfapp.corp.example.com         # expect 200 or 302 to login, valid cert
curl -I https://tfapp.corp.example.com/static/css/main.css
journalctl -u tfapp -n 50 --no-pager
```

`check --deploy` audits exactly the settings this migration changes, so treat any remaining warning as a real finding.

Then in a browser, because these paths are what actually break in a move like this:

- Log in and confirm styling loads, proving `/static/` resolves.
- Open the employee handbook PDF and view a profile photo, proving `/media/` resolves. This is the behaviour that does not work today.
- Punch in from a kiosk device and confirm no PIN is requested, proving kiosk IP matching survived the subnet change.
- Confirm a punch timestamp shows correct local Detroit time, proving the timezone is right.
- Run a payroll or attendance report that reads historical data, proving the restore is complete.
- Submit a PTO request to confirm writes and approval notifications work.

Step 17: Cutover and rollback

1. Build and validate the VM completely while the old server keeps running. Everything above can be done with test data.

2. Schedule a short freeze, ideally between pay periods, and tell staff.
3. Stop the old app so no punches are written during the dump: `sudo systemctl stop gunicorn cloudflared`.
4. Take the final dump and media tarball per Step 5.
5. Drop and recreate the database on the VM, then reload the final dump so no test data lingers:

```
sudo -u postgres dropdb tfapp
sudo -u postgres createdb --owner=tfapp --encoding=UTF8 tfapp
sudo -u postgres pg_restore --no-owner --no-privileges --role=tfapp -d tfapp -v /tmp/tfapp-final.dump
sudo systemctl restart tfapp
```

6. Sync any media added since the earlier copy, then update kiosk IP rows and work the Step 16 checklist.
7. Keep the old server powered off but intact for at least one full pay cycle. That is your rollback: power it on and re-enable cloudflared.
8. Only after that, decommission the tunnel and remove the external Cloudflare DNS record.

Optional repo cleanups

None of these are required for the migration, but each removes a footgun and would make this server reproducible:

- Replace the hardcoded `192.168.1.192` defaults in `DJANGO_ALLOWED_HOSTS` and `DJANGO_CSRF_TRUSTED_ORIGINS` in tfapp/tfapp/settings.py with the new host.
- Make the insecure `SECRET_KEY` fallback a hard error when `DEBUG` is false, so a misconfigured deploy refuses to boot instead of running on a published key.
- Commit the systemd unit and nginx config created here into the repo.
- Add the five `DB_*` variables to .env.example, which currently documents only `POSTGRES_*` and so does not actually work.